

Automatic Music Genre Classification

Authors: *Berta Miguel Ninou, Núria Esquiús Bau*

Date: *June 2026*

1. Introduction

Music genre classification means automatically deciding what genre a song belongs to, whether it is blues, rock, jazz or any of the other ten categories we work with. This kind of task matters a lot in practice: streaming platforms like Spotify have millions of songs and need to organize them so that recommendations and content filtering can work properly. Doing this by hand would be extremely slow, expensive and would produce inconsistent results because genre is partly subjective.

Deep learning gives us a way to solve this automatically. Instead of writing rules about what makes a song sound like jazz or metal, we let the model discover those patterns on its own by learning from examples. The patterns it finds are things like rhythm, texture of the sound (timbre) and the harmonic content of the audio.

In this project we compare three different deep learning approaches on the same dataset:

- A Deep Neural Network (DNN) that works with 57 hand-crafted numerical features extracted from each audio clip.
- A Convolutional Neural Network (CNN) that reads the audio as a visual image Spectrogram.
- A fine-tuned ResNet18, which is a powerful pre-trained image model that we adapt to our spectrogram images.

We use the GTZAN dataset, which is the standard benchmark in Music Genre Recognition research. It contains 1,000 labelled audio clips across 10 genres.

2. State of the Art

Research on automatic music genre recognition started in the early 2000s. Tzanetakis and Cook (2002) introduced the GTZAN dataset and proposed using a type of audio feature called Mel-Frequency Cepstral Coefficients (MFCCs) together with simple classifiers, which set the baseline for all subsequent work. Since then, the field has gone through several stages:

- Traditional machine learning (2000s): Models like Support Vector Machines (SVMs) and Random Forests were used with hand-crafted features. They reached around 70-80% accuracy on GTZAN.
- CNNs on spectrograms (2014 onwards): By converting audio into spectrogram images and treating them as photos, convolutional networks could learn features automatically. This improved accuracy up to around 85-90%.
- Transfer learning (2017 onwards): Models originally trained on millions of images (like VGG and ResNet) were fine-tuned on spectrograms. This improved performance, especially when training data is limited.
- Transformers (2021 onwards): Audio-specific transformer models such as the Audio Spectrogram Transformer (AST) represent the current state of the art, but they require massive amounts of data and computing resources to train.

Our project sits in the CNN + Transfer Learning category, which is a solid and well tested approach for datasets of the size of GTZAN.

3. Methodology

3.1 Data Analysis

Dataset

The GTZAN dataset contains 1,000 audio clips, 100 per genre across 10 classes: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae and rock. Each clip is 30 seconds long and recorded at 22,050 Hz. The dataset is balanced, meaning every class has the same number of samples.

The dataset comes in two formats that we use in different ways:

Format	Used for	Samples
images_original/ (.png)	CNN, ResNet18	1,000 (one per clip)
features_3_sec.csv	DNN	9,990 (10 fragments × 1,000 clips)
features_30_sec.csv	DNN	1,000

Key design choice: 3-second fragmentation

For the DNN, instead of using the 1,000 full 30-second clips, we cut each clip into 10 three-second pieces and extract features from each piece. This multiplies our training data by 10, giving us 9,990 samples instead of 1,000. Three seconds is still long enough to contain useful information about rhythm, timbre and harmonic patterns. Having more training examples is critical for neural networks to generalise well, so this fragmentation is one of the most important design decisions in the project. We also ran a comparison with the 30-second version to see how much this choice mattered.

Feature analysis

The CSV files contain 57 features per clip. These include 40 MFCC (Mel-Frequency Cepstral Coefficients) values, the mean and variance for 20 MFCCs, plus chroma, spectral centroid, spectral bandwidth, rolloff, zero crossing rate, RMS energy, harmony and tempo. We discard the filename and length columns as they contain no useful information for classification.

MFCCs are the most important features because they capture the timbral texture of sound, what an instrument or voice 'sounds like'. Tempo is also important: classical music tends to be around 70 BPM while metal can reach 180 BPM.

Preprocessing

Each model needs its input in a specific format, so we preprocess differently for each one:

Model	Input	Preprocessing
DNN	57 features	StandardScaler (zero mean, unit variance)
CNN	Spectrogram 128×128	Resize + Normalize (mean=0.5, std=0.5 per channel)
ResNet18	Spectrogram 224×224	Resize + ImageNet normalisation (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

We use an 80-10-10 split (train / validation / test). The same indices are used across all three models so the comparison is fair. The split is stratified, meaning each class has the same proportion in every subset.

We intentionally did not apply data augmentation (like flipping or shifting the spectrograms) because the axes of a spectrogram encode real physical information, time and frequency. Distorting those would destroy the meaning of the image.

3.2 Models and Optimization

MLP: Multi-Layer Perceptron

We use a Multi-Layer Perceptron (MLP) that takes the 57 numerical features as input and outputs a probability score for each of the 10 genres.

The architecture is: 3-layer MLP: $57 \rightarrow 256 \rightarrow 128 \rightarrow 10$.

Architecture: starting wide and getting narrower is a common and effective choice for classification. The first layer (256 neurons) gives the model enough capacity to detect complex combinations of features. The second

layer (128 neurons) compresses this into a more compact representation. The final layer (10 neurons) produces one score per genre. Going deeper than three layers would not help much because we only have 57 input features.

ReLU: Without non-linear activations, multiple linear layers would collapse into a single linear transformation. ReLU introduces non-linearity while being very efficient to compute and avoiding the vanishing gradient problem that other activations like sigmoid gives.

Dropout(0.3): Dropout randomly switches off 30% of the neurons during each training step. This forces the network to learn redundant representations. The result is a model that generalizes better to new data. We chose 0.3 as a moderate value: strong enough to generalize, but not too aggressive that it slows the learning.

Training: Adam optimiser with learning rate 1e-3, CrossEntropyLoss, 100 epochs, batch size 32. Adam is a good default choice because it adapts the learning rate for each parameter automatically, making it less sensitive to the initial learning rate than plain SGD. 100 epochs was enough for the loss to converge. CrossEntropyLoss is the standard choice for multi-class classification.

CNN: Convolutional Network

The CNN reads the Mel Spectrogram of each audio clip as a 128×128 pixel image with 3 colour channels. Instead of using handcrafted features, it learns to recognise visual patterns in the spectrogram directly from the pixels.

The architecture has three convolutional blocks followed by a classifier:

Block 1: Conv2d($3 \rightarrow 32$ filters, 3×3 kernel) + BatchNorm + ReLU + MaxPool(2×2)

Block 2: Conv2d($32 \rightarrow 64$ filters, 3×3 kernel) + BatchNorm + ReLU + MaxPool(2×2)

Block 3: Conv2d($64 \rightarrow 128$ filters, 3×3 kernel) + BatchNorm + ReLU + MaxPool(2×2)

AdaptiveAvgPool $\rightarrow$ Dropout $\rightarrow$ Linear($128 \rightarrow 10$)

Three convolutional blocks: Each block captures patterns at a different scale. The first block learns simple local patterns like edges and basic textures. The second block combines those into intermediate patterns. The third block identifies higher-level structures. With only 800 training images, going deeper would risk overfitting.

Doubling filters ($32 \rightarrow 64 \rightarrow 128$): This is a standard CNN design pattern. As the spatial resolution shrinks (MaxPool), we increase the number of filters to maintain the information of the representation. More filters means the network can describe more types of patterns at each scale.

BatchNorm: Batch Normalisation stabilizes training by normalizing the activations within each batch. This lets us use a higher learning rate and makes the network less sensitive to weight initialization.

128×128 input: This is a good balance between resolution (keeping enough detail to distinguish genres) and computational cost. The original spectrograms vary in size, so resizing to a fixed size is necessary.

Training configuration: Adam optimiser, learning rate 1e-3, CrossEntropyLoss, 30 epochs (training curves showed convergence well before 30). The model is saved at the epoch with the lowest validation loss.

ResNet18: Transfer Learning

ResNet18 is a convolutional network originally trained on ImageNet, a dataset with 1 million images across 1,000 categories. It has 11 million parameters and has already learned to detect edges, textures and complex visual patterns.

We replace only the final fully connected layer with a new layer that outputs 10 scores, one per music genre. Then we fine-tune different amounts of the network.

We tried four fine-tuning strategies, unfreezing progressively more layers:

- Fully Connected (fc) only: only the final layer is trained. The rest stays frozen.
- Layer4 + fc: the last residual block and the final layer are trained.

- Layer3 + Layer4 + fc: the two deepest residual blocks and the final layer are trained.
- Layer2 + Layer3 + Layer4 + fc: three residual blocks and the final layer are trained.

Different learning rates: The deeper layers (closer to the output) need to adapt more to spectrograms, so they get a higher learning rate ($1e-4$ for fc, $1e-5$ for residual blocks). The earlier frozen layers already contain useful generic features (edges and textures).

ImageNet normalisation for ResNet: The pretrained weights were computed on images normalized with ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]). Using different normalisation would create a mismatch and hurt the transfer.

224×224 for ResNet and 128×128 for the custom CNN: ResNet18 was designed for 224×224 inputs. Our custom CNN does not have this constraint, so we use 128×128 to reduce the computational cost.

4. Experiments

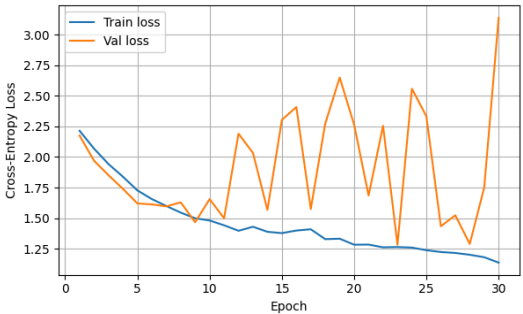
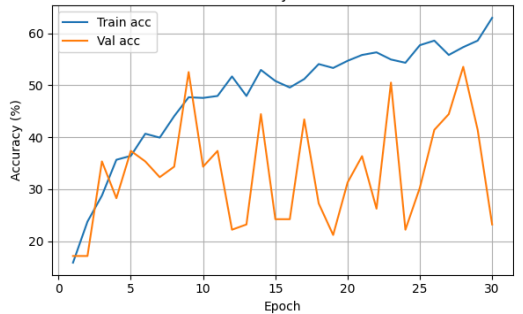
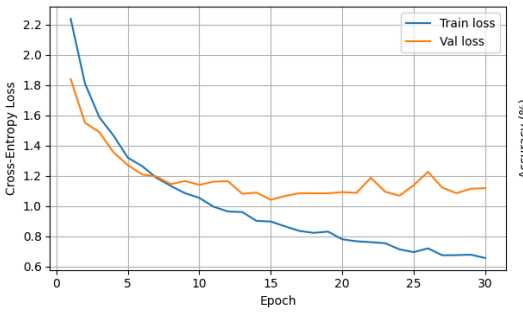
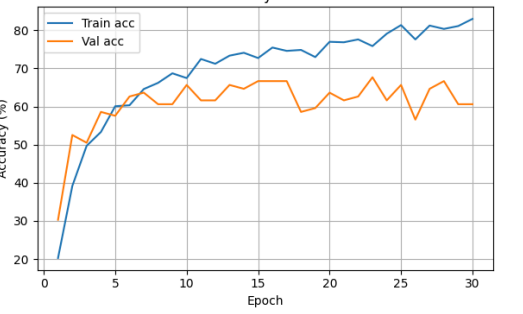
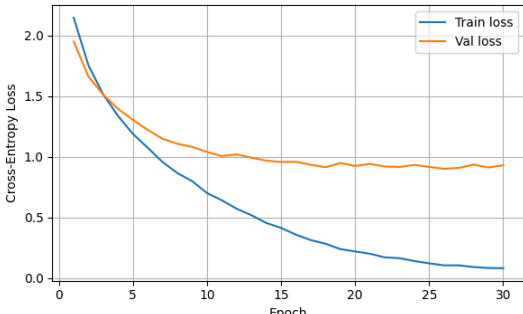
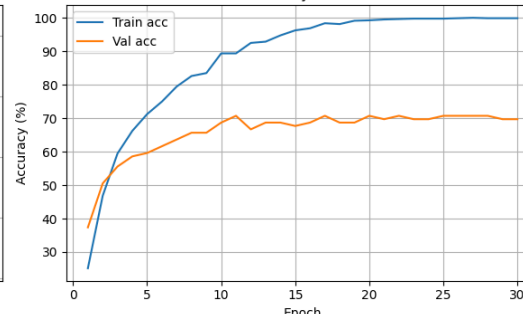
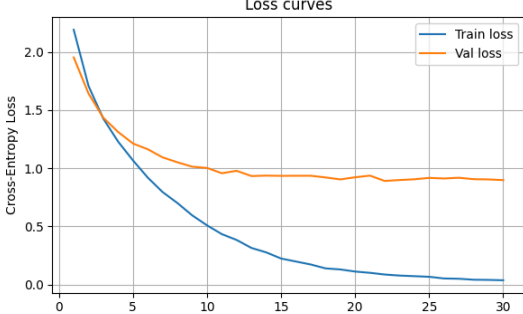
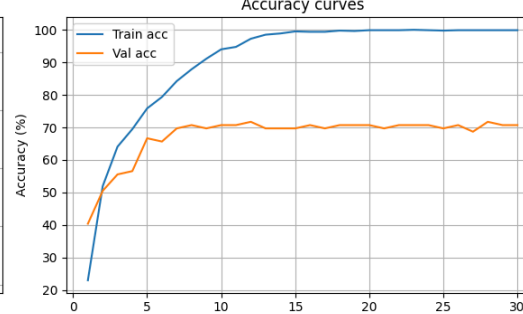
4.1 Experimental Setup

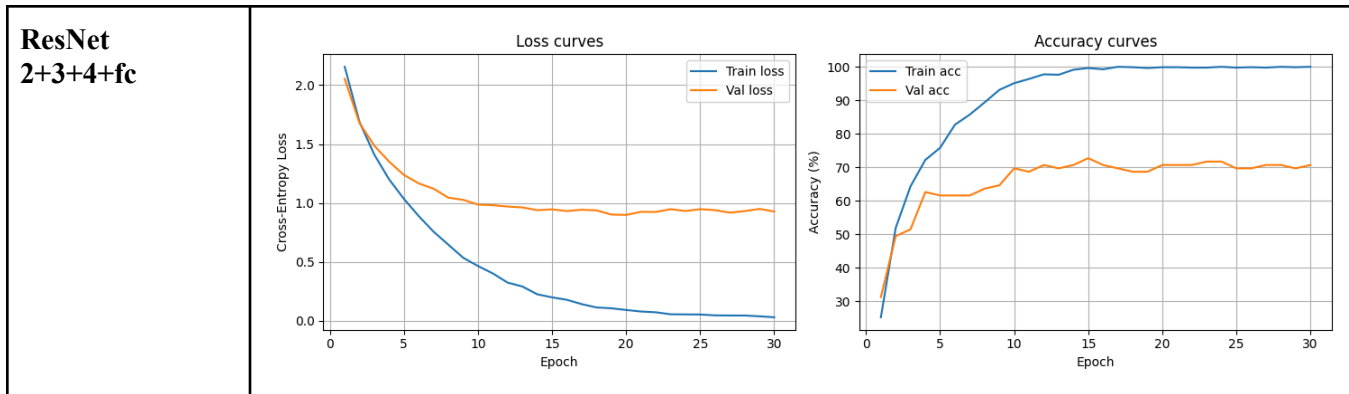
- Hardware: Google Colab (GPU runtime)
- Framework: PyTorch
- Random seed: 42 (reproducibility)
- Evaluation metric: Accuracy on test set (10%)
- Cross-validation: Stratified 5-Fold on MLP to validate robustness and reliability

4.2 Training Curves

These are the different train and validation curves, both for loss and accuracy.

Models	Loss curves	Accuracy curves
MLP (3s features)		
MLP (30s features)		

CNN	Loss curves  Accuracy curves 
ResNet fc	Loss curves  Accuracy curves 
ResNet 4+fc	Loss curves  Accuracy curves 
ResNet 3+4+fc	Loss curves  Accuracy curves 



The training curves show important differences in learning across models. Below we analyse each model.

MLP (3s features): Both training and validation loss decrease smoothly over 100 epochs with no sign of overfitting. Validation accuracy grows to around 76%, finishing slightly above training accuracy, a correct result that confirms the model is generalizing well. The smooth convergence also confirms that 100 epochs was an appropriate training length.

MLP (30s features): The 30-second MLP shows a similar smooth convergence as the 3-second version, but with a more big gap between training and validation accuracy in the later epochs. Training accuracy is high (~80%+) while validation accuracy stabilizes around 70–75%. This gap indicates overfitting. This makes sense: with only 800 samples (ten times fewer than the 3-second version), the model has less variety to learn from. The loss curves still converge cleanly, which confirms that 100 epochs is appropriate, but the generalization is weaker.

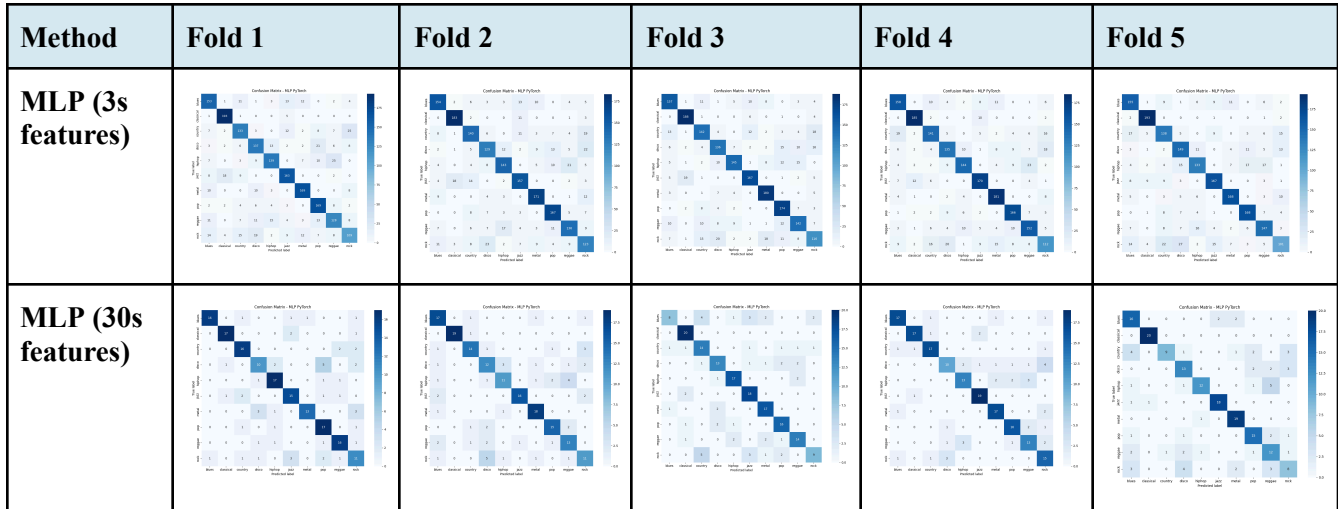
CNN: The curves are clearly noisy. Validation loss oscillates significantly across epochs, which is a clear sign of instability caused by training on only 800 images. Despite this noise, the overall trend shows improvement and the model is saved at its best validation checkpoint.

ResNet (fc only): When only the fully connected layer is trained, the curves show a moderate and stable improvement. Validation accuracy increases from 30% to around 62–65%, while training accuracy also stabilizes around the same range. The small gap between training and validation performance is the key observation: because almost all layers are frozen, the model cannot overfit. Training only the fc layer without any adaptation to spectrogram structure is not sufficient to improve ~62% accuracy.

ResNet18 (Layer3+4+fc strategy): This is the strongest ResNet variant. Validation accuracy increases fast in the first 5 epochs (from ~20% to ~65%) then continues to improve more slowly, around 70–72%. Training accuracy reaches near 100%. The gap between training and validation performance is large, clear sign of overfitting with 800 images, but the validation accuracy is higher than the other ResNet strategies. The explanation is that Layer3 and Layer4 are the most “task-specific” layers in ResNet18: by unfreezing them, the model can adapt mid-level and high-level visual features to the particular structure of spectrograms. Unfreezing Layer4+fc or Layer2+3+4+fc give us a very similar loss curve and accuracy curve, however the accuracy curves are not as stable and better than Layer3+4+fc.

4.3 Cross-Validation Results: MLP

To validate that MLP results are not dependent on a specific split, we applied Stratified K-Fold (K=5). Each fold uses a different 80/20 partition while maintaining class balance.



None RESULTS 5-FOLD (3s-features) Fold 1: 0.7472 Fold 2: 0.7523 Fold 3: 0.7743 Fold 4: 0.7728 Fold 5: 0.7603 Mean: 0.7614 Std: 0.0108 Final result: 0.7614 ± 0.0108	None RESULTS 5-FOLD (30s-features) Fold 1: 0.7400 Fold 2: 0.7300 Fold 3: 0.7300 Fold 4: 0.7700 Fold 5: 0.7100 Mean: 0.7360 Std: 0.0196 Final result: 0.7360 ± 0.0196
---	--

Every fold of the 3s version outperforms the corresponding fold of the 30-second version. This is not a coincidence, it demonstrates that the data fragmentation provides a reliable benefit, not just a lucky split. For the 3s MLP the std deviation drops from $\pm 1.96\%$ (30-second) to $\pm 1.08\%$ (3-second). A lower std deviation means the models performance is less dependent on which samples fall in the training set, o 3s model is more reliable. The 30s version has a lower finally average accuracy, that suggests that with only 800 samples, certain test splits can significantly hurt performance. With more samples this risk is reduced.

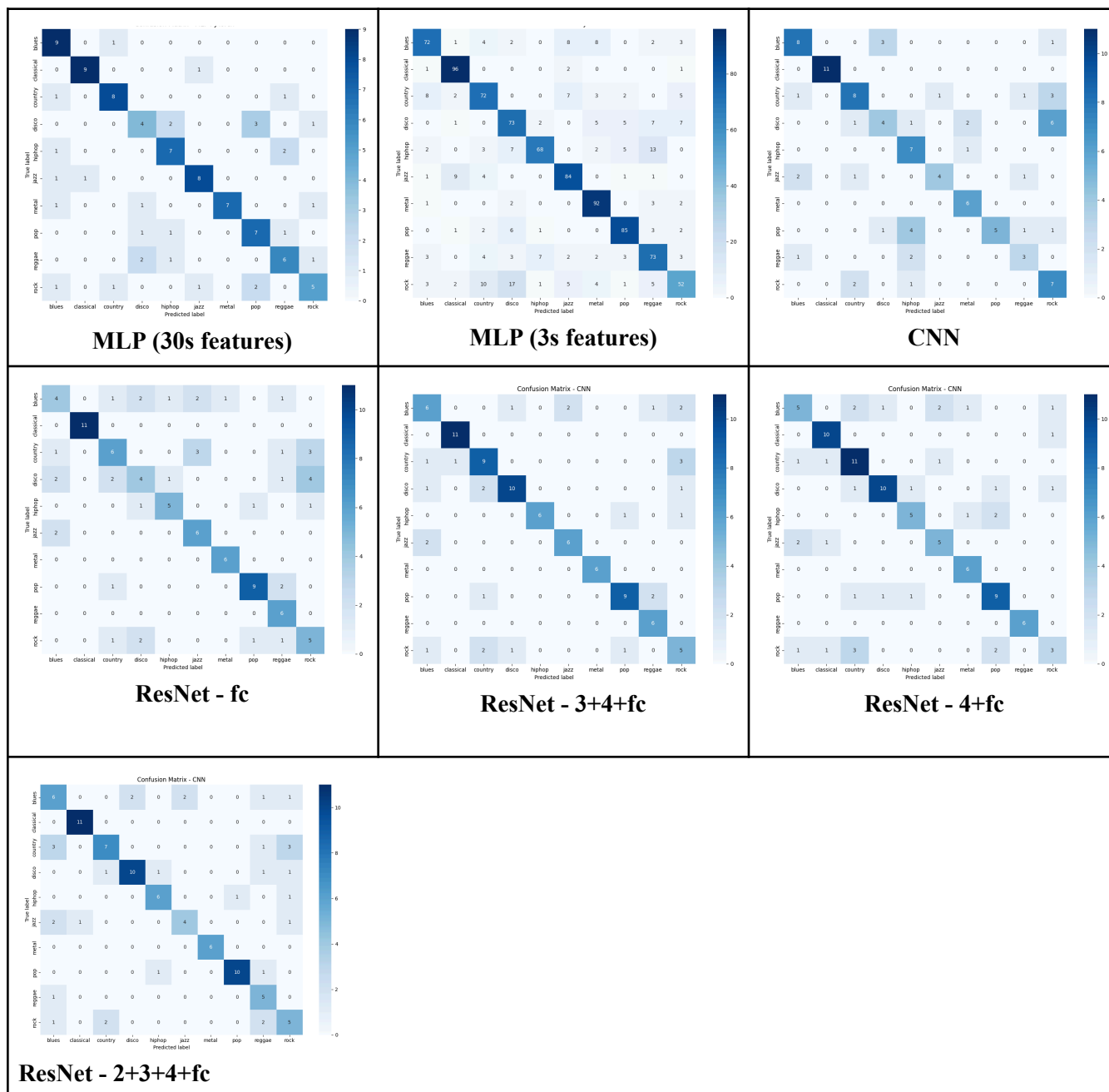
5. Results

5.1 Overall Accuracy Comparison

Model	Input data	Train samples	Test Accuracy
MLP	57 features (3s CSV)	~7992	76.18%
MLP (5-fold)	57 features (3s CSV)	~7992	76.14% ± 0.01
MLP	57 features (30s CSV)	800	70%
MLP (5-fold)	57 features (30s CSV)	800	73.60 ± 0.019
CNN	Mel Spectrograms 128×128	800	~60%

ResNet18 (fc)	Mel Spectrograms 224×224	800	62.38%
ResNet18 (Layer4 + fc)	Mel Spectrograms 224×224	800	65.35%
ResNet18 (Layer3+4+fc)	Mel Spectrograms 224×224	800	72.28%
ResNet18 (Layer2+3+4+fc)	Mel Spectrograms 224×224	800	68.32%

5.2 Confusion Matrices



MLP - 30s: With only 800 training samples, the 30s MLP shows more diffuse confusion patterns. Rock is the hardest to classify, with many samples misclassified with country, disco, hip-hop and jazz. Blues is frequently confused with jazz and country. The diagonal of the matrix is less dark compared to the 3-second version, indicating weaker per-class performance. Classical and metal are the ones classified better, suggesting these genres have the most distinct sound.

MLP - 3s: The 3-second MLP shows a much stronger and cleaner diagonal. Classical (96/100 correct), metal (92/100), jazz (84/100) and pop (85/100) are classified with high confidence. Rock remains the hardest class (52/100), getting confused mainly with country, disco and hip-hop. The improvement over the 30-second version is visible in the matrix: the off-diagonal elements are much smaller, meaning the model makes fewer and less mistakes.

CNN: Classical and metal are classified perfectly (11/11 and 6/6). Disco is very problematic, with confusion spread across hip-hop, metal and rock, the CNN struggles with genres with similar patterns. The matrix is generally sparser but noisier than the MLP matrices.

ResNet18 - Layer3+4+fc: This matrix shows the best performance across all ResNet variants. Pop is classified perfectly (12/12). Classical (10/11), country (10/14) and metal (6/6) are also very strong. Blues and rock remain hard to classify (6/12 and 4/10). Compared to the fc-only matrix, the diagonal is darker, confirming that adapting the deeper layers provides improvement in genre classification.

6. Discussion

6.1 Key findings

MLP vs CNN: The MLP had access to 7992 training samples while the CNN only had 800 images. In deep learning, more training data is often more important than a better architecture. On top of that, the 57 features the MLP uses encode expert knowledge about music that took decades of research to develop, the CNN has to rediscover equivalent patterns from scratch.

ResNet18 vs CNN: Both models see the same 800 training images. The difference is that ResNet18 already knows how to detect edges, textures and shapes from having been trained on 1 million images. Some of that knowledge transfers to spectrograms, even though spectrograms are not natural images. This explains why ResNet18 (72.28%) outperforms the CNN (~60%), despite having the same amount of training data.

ResNet fine-tuning strategies: Unfreezing more layers doesn't always help. Layer3+4+fc (72.28%) beats Layer2+3+4+fc (68.32%) because unfreezing too many layers with a small dataset leads to overfitting, the model starts adapting the parts that were already working well. Layer3 and Layer4 are the most "task-specific" layers, so adapting those is beneficiary.

Both the CNN and all ResNet have the same problem: 800 training images are not enough. The training curves show near 100% training accuracy and much lower validation accuracy. The MLP avoids this problem because it uses 9,990 samples. For spectrogram models to compete and achieve similar results to MLP, they would need much more training data or stronger regularisation (e.g., bigger dropout, weight decay, early stopping).

The confusion patterns we observe in the matrices are not random errors. Rock and metal share guitar sounds and high energy. Blues and jazz share harmonics and instrumentation. Disco and reggae share rhythmic patterns. The fact that the deep learning models make the same confusions as a human would means that the models are learning real acoustic features.

6.2 Limitations

- GTZAN contains only 1,000 clips, which is small for deep learning. Models trained on this data may not generalise well to music outside the dataset.

- The 3-second fragmentation creates a data leakage risk: fragments from the same 30-second song appear in both the training and test sets. This may increase the DNN accuracy slightly, because fragments from the same song can sound similar.

7. Conclusions

This project compares three Deep Learning approaches to music genre classification on the GTZAN benchmark. The key insight is that data quantity and input representation matter as much as architectural complexity: the DNN, despite its simpler architecture, benefits from 10x more training samples through 3-second fragmentation, and the ResNet18 compensates for limited data through transfer learning.

The results confirm this clearly. The 3-second MLP achieves 76.14% accuracy ($\pm 1.08\%$ across 5 folds), outperforming the 30-second MLP (73.60% $\pm 1.96\%$) and demonstrating that fragmentation not only increases accuracy, but also improves model stability. The CNN, limited to 800 training images, struggles to generalize, its training curves show near 100% training accuracy with much lower validation accuracy, a clear sign of overfitting. ResNet18 partially overcomes this limitation through transfer learning, reaching 72.28% by adapting layers 3 and 4.

The confusion matrix analysis shows that model errors are not random, they reflect real acoustic similarities. Rock and metal share guitar distortion and high energy. Blues and jazz share harmonics and instrumentation. Disco and reggae share rhythmic patterns. The fact that our models make the same confusions a human expert suggest is learning acoustic features rather than false correlations.

For future work, one important improvement would be increasing the spectrogram training data. Stronger regularization (larger dropout, weight decay, early stopping) could also help the CNN and ResNet models generalize better from limited data. Finally, exploring audio architectures such as Audio Spectrogram Transformers (AST).

8. References

- Tzanetakis, G., & Cook, P. (2002). Musical genre classification of audio signals. *IEEE Transactions on Speech and Audio Processing*, 10(5), 293–302.
- Sigtia, S., & Dixon, S. (2014). Improved music feature learning with deep neural networks. *ICASSP 2014*.
- Choi, K., et al. (2017). Transfer learning for music classification and regression tasks. *ISMIR 2017*.
- Gong, Y., et al. (2021). AST: Audio Spectrogram Transformer. *Interspeech 2021*.
- Andradaolteanu. (2020, March 23). GTZAN Dataset - Music Genre Classification. Kaggle. <https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>
- Rosebrock, A. (2017, March 19). ImageNet: VGGNet, ResNet, Inception, and Xception with Keras. PyImageSearch. <https://pyimagesearch.com/2017/03/20/imagenet-vggnet-resnet-inception-xception-keras/>
- GeeksforGeeks. (2021, April 15). How to normalize images in PyTorch? GeeksforGeeks. <https://www.geeksforgeeks.org/python/how-to-normalize-images-in-pytorch/>
- PyTorch. (n.d.). torch.nn — PyTorch 2.12 documentation. <https://docs.pytorch.org/docs/2.12/nn.html>
- GeeksforGeeks. (2020, October 21). Introduction To Adam Optimizer. <https://www.geeksforgeeks.org/deep-learning/adam-optimizer/>
- Deep Learning Course, Class 9: Convolutional Neural Networks. UPF, 2026.